

Team Reflections - CDMP (Canadian Donation Map)

Team NACSS: Calvin Liew, Shahveer Rasool, Shamrita Saravanakumar, Abdul Hameed Mohammed, Niveetha Sivakaran

1. Features Not Completed

We did not complete two features we had originally considered: a standalone “Donation Intelligence” analytics dashboard, and an email-based account verification / password-reset (“forgot password”) flow.

Why we dropped it: By the time we scoped this feature, each province and riding already had a live summary panel, and the research page featured an analytics panel (top ridings by donation count, donation amount concentration/distribution) alongside the filtered records table. Between the map’s per-riding summary, the trend view, and the research analytics panel, we already surfaced most of the insight this dashboard would have added, so a separate, deeper analytics layer felt redundant. We chose to drop it rather than build a feature that would mostly restate information the user could already see elsewhere in the app. In addition, we had decided to introduce new metrics through the newly built AI agent, “Ask CDMP”.

We also dropped the email-based account verification and password-reset (“forgot password”) feature. Sending a verification or reset email requires wiring the application to a third-party email service, for example Ethereum or SendGrid, which meant taking on another external dependency, a set of credentials to manage, and additional failure modes to test and document. We judged that the effort and added operational complexity outweighed the value for a course project whose core audience is researchers registering with a recognized university email address, and we chose to spend that time finishing and hardening Ask CDMP instead.

How we chose what to drop: We prioritized based on (a) how much new insight or value a feature would add over what already existed, and (b) direct feedback from our professor. The analytics dashboard largely duplicated views we had already shipped, and the email flow added third-party complexity for limited benefit in our context. Meanwhile, our professor’s feedback pointed us toward adding a conversational way to query the data, so we redirected the time we would have spent on both features into building the Ask CDMP AI assistant, which we felt gave users, especially less technical ones, a more valuable and differentiated way to interact with the donation data than another set of charts or an email pipeline would have.

2. Software Engineering Practices

What we adopted

- Feature branches merged into develop/main via pull request, rather than committing directly to main.
- Continuous integration (GitHub Actions) running the Jest test suites for both server and client on every push/PR to develop and main.

- Automated test suites split by concern: backend (Jest + Supertest) with the database mocked, frontend unit tests (Jest) with fetch mocked, and Cypress end-to-end tests against a running Docker instance.
- A documented Docker-based handoff (DOCKER.md) so the app, and a restorable database snapshot, could be run identically by any teammate or grader without manually rebuilding the ~5.3 million-row dataset.
- Periodic external/self code-review passes (e.g. our Sprint 4 review) that re-audited the codebase against acceptance criteria rather than only relying on the original author's judgment.

What was effective: The mocked, isolated unit tests were fast and effective for catching regressions in routes and API clients without needing infrastructure, and the Docker + Mongo-dump handoff meant we (and eventually a grader) could stand the whole system up in minutes instead of re-running a multi-step data pipeline. Feature branching also made it much easier to keep main in a demoable state.

What was less effective: Our CI only ran automated tests, not lint or the production build, and coverage had gaps around newer endpoints (e.g. account info, password change, and the Ask CDMF endpoint) until a dedicated review pass caught them. Because code quality checks weren't integrated in CI, code quality issues accumulated in newer code without blocking merges, which meant we caught some issues later than we would have liked. We also found, during that same review, a privacy-agreement bypass in the authentication flow that hadn't been caught by any of our existing test suites, which reinforced that automated tests were necessary but not sufficient.

Adapting as the project progressed

We adapted our review process over the course of the project. Early on, we relied mostly on pull-request review by a teammate, but as the codebase grew (particularly around authentication and the research dashboard) we moved to a more structured, checklist-style review pass that explicitly re-validated security-sensitive flows (privacy agreement, access control) and cross-checked that different parts of the app (e.g. table filters vs. analytics filters) agreed with each other. We also shifted scope mid-project: once we saw that our planned analytics dashboard overlapped heavily with functionality we'd already shipped, and that the email verification flow would add third-party complexity for limited benefit, we reallocated that time toward the Ask CDMF AI assistant based on professor feedback, rather than building features we felt would add limited additional value.

What we would do differently next time

- Add code quality checks and the production build as required, blocking CI checks from the start, not just automated tests.
- Write tests for new endpoints in the same PR that introduces them, rather than relying on a later review pass to catch gaps.
- Do a dedicated security/consistency review earlier (e.g. after the authentication flow first lands) instead of only at a later sprint checkpoint.

- Settle on our MongoDB vs. local/Dockerized environment strategy earlier, since switching between them cost us real time (see Section 3).
- Decide earlier which features carry an external dependency (such as an email provider), so we can commit to or cut them before investing partial effort.

3. Technical/Design Challenges

What we're proud of

We're particularly proud of three things working together. The first is Ask CDMP, our AI assistant, which lets users ask questions about the donation data in plain language instead of manually constructing filter combinations. What we're most proud of is not that it "uses AI," but the constrained design around it: a natural-language question is translated by the model into a restricted, validated QuerySpec, the backend then executes only approved aggregate operations, and results are privacy-suppressed before they are returned. The assistant is provider-agnostic (it works with OpenAI-compatible models, Anthropic Claude, or Google Gemini through a single adapter), answers totals, rankings, trends, comparisons, and year-over-year changes, supports conversational follow-ups, and can apply its answer straight back onto the map. Grounding the model in our actual data and refusing anything outside that QuerySpec is what makes the feature trustworthy rather than a generic chatbot. The second is the research login's export and expanded statistics, which let authenticated researchers pull individual-level CSV records and see summary analytics (top ridings, donation concentration) alongside them without leaving the dashboard. The third is the boundary-vintage filtering on the riding map, which lets a user look at riding boundaries as they actually existed for a given date range rather than overlaying modern boundaries onto historical data.

Most difficult technical challenge

The most difficult technical challenge was standardizing our MongoDB setup and Dockerizing the application, and being able to move cleanly between a local MongoDB instance (for day-to-day development) and a fully Dockerized stack with its own containerized MongoDB (for handoff, demos, and grading). Early on, differences between how data was seeded/imported locally versus inside Docker containers caused inconsistent state between teammates' machines and made it hard to reproduce issues. We addressed this by settling on a single documented data-handoff path: a Docker Compose stack with its own MongoDB container, a restorable Mongo dump distributed via Google Drive (kept out of GitHub due to size), and a documented full-rebuild path (import and aggregation scripts) for when the dump alone wasn't enough to validate the pipeline. Building Ask CDMP was the other major challenge, and getting it fully shipped meant solving two hard problems: scoping what it should be allowed to answer, and grounding its responses in the actual donation data rather than letting the model answer from general knowledge. We solved both with the validated-QuerySpec pipeline described above, deterministic safeguards for common supported questions and riding-name normalization, and coverage notes for known data gaps. It took a lot of iteration to get right, but it ended up being one of the most valuable features we added, especially for less technical users who don't want to think in terms of filters.